01

Part 1 · Spark Foundations

How Spark is built and how a query becomes work on a cluster: the mental model everything else rests on.

- Architecture: driver · executors · cluster manager
- RDD vs DataFrame vs Dataset
- Lazy evaluation
- Transformations & actions
- DAG
- Jobs · stages · tasks
- Logical & physical plans
- Catalyst & Tungsten
- Spark UI

1.1 Spark Architecture: Driver, Executors, Cluster Manager

★★★

IN ONE LINE

A driver plans and schedules; executors on worker nodes run tasks on partitions in parallel.

Apache Spark is a distributed engine that splits data into **partitions** and processes them in parallel on many machines. To tune it you must know who does what.

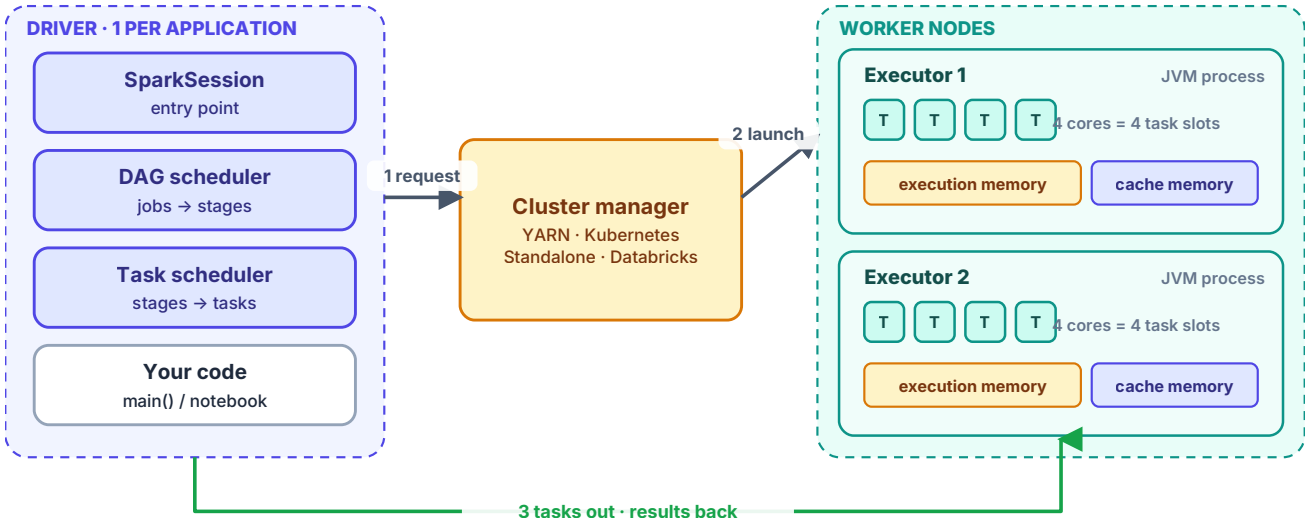


Figure 1.1 — Driver, cluster manager and executors

Component	Role	Why it matters for tuning
Driver	Runs your `main()`, builds the plan, schedules tasks, collects results.	Too much data sent to it (`collect()`, big broadcasts) causes driver OOM.
Cluster manager	Hands out machines/containers (YARN, Kubernetes, Standalone, Databricks).	Decides how many executors you actually get.
Executor	A JVM process on a worker that runs tasks and holds cached data.	Its cores and memory limit parallelism and spill.
Core / slot	One task runs on one core at a time.	Total cores = max parallel tasks.
Task	Unit of work on **one partition**.	Number of tasks = number of partitions (per stage).

Deploy modes

Client: the driver runs where you submit (notebook, laptop). **Cluster**: the driver runs inside the cluster (production jobs).

Spark vs MapReduce

Spark keeps intermediate data in **memory** and pipelines operations, so iterative and interactive work is far faster than disk-based MapReduce.

INTERVIEW ANGLE

A classic opener: *"Explain Spark architecture."* Name **driver, cluster manager, executors, tasks**, say that parallelism = partitions × cores, and mention that the driver is a single point that must not be overloaded.

1.2 RDD vs DataFrame vs Dataset

★★★

IN ONE LINE

DataFrames add a schema so Spark can optimise; RDDs are the unoptimised low-level base.

Spark has three data abstractions. Interviews expect you to know why **DataFrames** are the default: they carry a **schema**, so Spark can optimise the whole query.

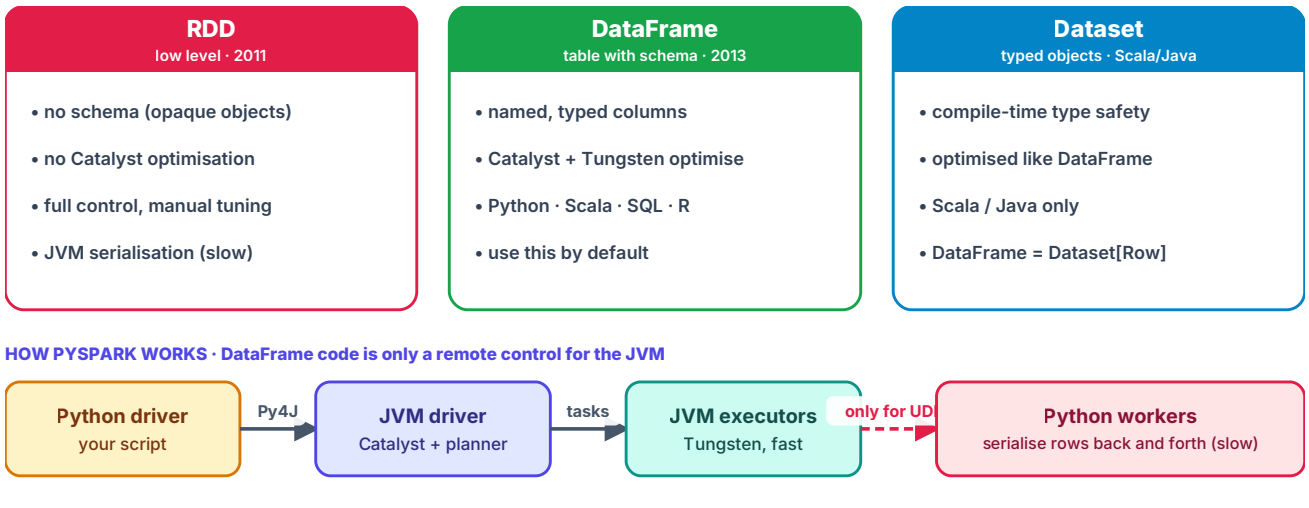


Figure 1.2 — RDD vs DataFrame vs Dataset, and how PySpark talks to the JVM

	RDD	DataFrame	Dataset
Optimiser (Catalyst)	No	Yes	Yes
Memory format	Java objects	Tungsten binary (compact)	Tungsten binary
Type safety	Compile time	Runtime	Compile time
PySpark available	Yes	**Yes**	No
Typical use today	Rare: custom low-level logic, unstructured data	**Almost everything**	Typed Scala / Java apps

SAME OPTIMISED ENGINE BEHIND EVERY API

```
df = spark.read.parquet('/lake/silver/orders')      # DataFrame
rdd = df.rdd                                          # drops to RDD: avoid unless needed
df.createOrReplaceTempView('orders')                  # same engine, now in SQL
spark.sql('SELECT city, count(*) FROM orders GROUP BY city')
```

COMMON TRAP

Converting a DataFrame to an RDD (`df.rdd`) and back discards Catalyst optimisation and Tungsten memory layout. Only do it when no DataFrame function can express the logic.

1.3 Lazy Evaluation

★★★

IN ONE LINE

Spark records transformations as a plan and only executes when an action asks for a result.

Lazy evaluation means Spark does **nothing** when you call a transformation such as `filter` or `select`. It only remembers the recipe (a plan). Work starts when an **action** (like `show`, `count`, `write`) demands a result.

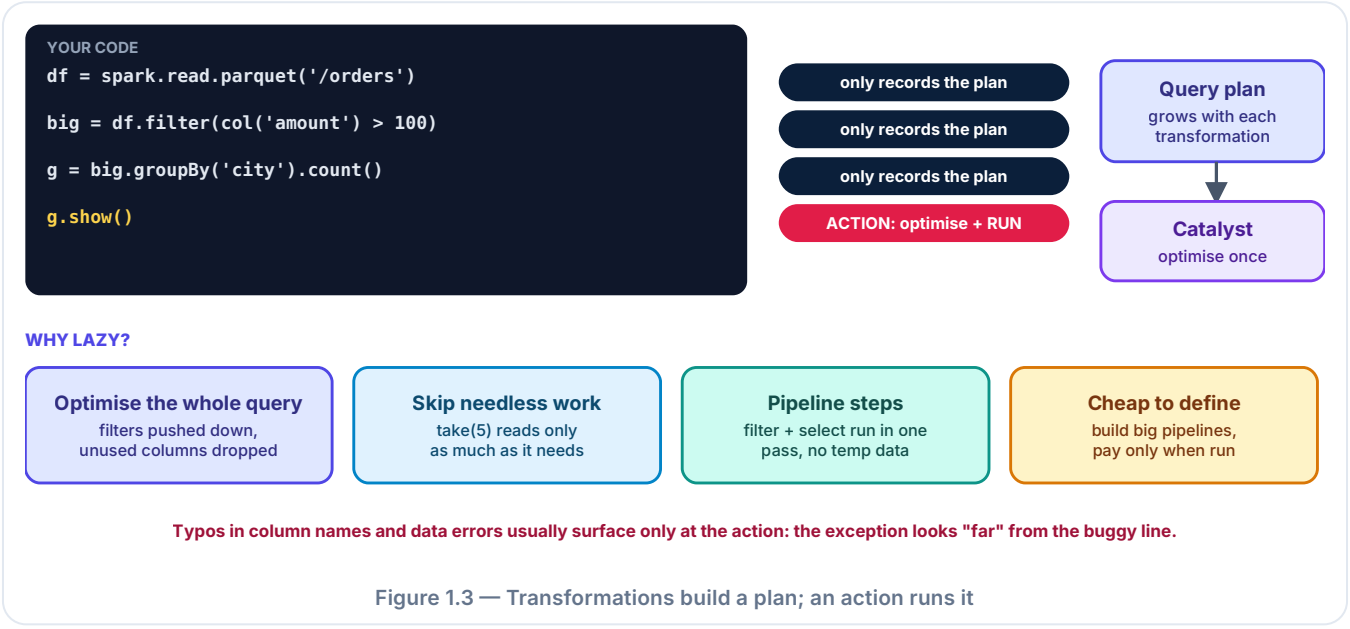


Figure 1.3 — Transformations build a plan; an action runs it

Benefits

- Spark sees the **whole** pipeline and optimises it as one unit.
- Only the data actually needed is read.
- Lost partitions can be recomputed from the recipe (lineage).

Side effects to know

- Each action **re-runs** the whole plan unless you `cache()` (4.1).
- `print` or timers around a transformation measure nothing.
- Errors appear at the action, not at the faulty line.

BENCHMARKS MUST INCLUDE AN ACTION

```
import time
t = time.time()
result = df.filter('amount > 100').groupBy('city').count()   # returns instantly
print(time.time() - t)                                       # ~0.01 s: nothing ran!
result.count()                                               # the real work happens here
```

REAL-WORLD ANALOGY

Ordering at a restaurant: you tell the waiter every dish (transformations) but the kitchen starts cooking only when you say “that is all” (action). The chef can then plan the whole meal efficiently.

1.4 Transformations and Actions

★★★

IN ONE LINE

Transformations lazily define new DataFrames; actions run the plan and return or write a result.

Transformations describe how to derive a new dataset; they are lazy and never change the original (DataFrames are **immutable**). **Actions** return a result to the driver or write output, and force execution.

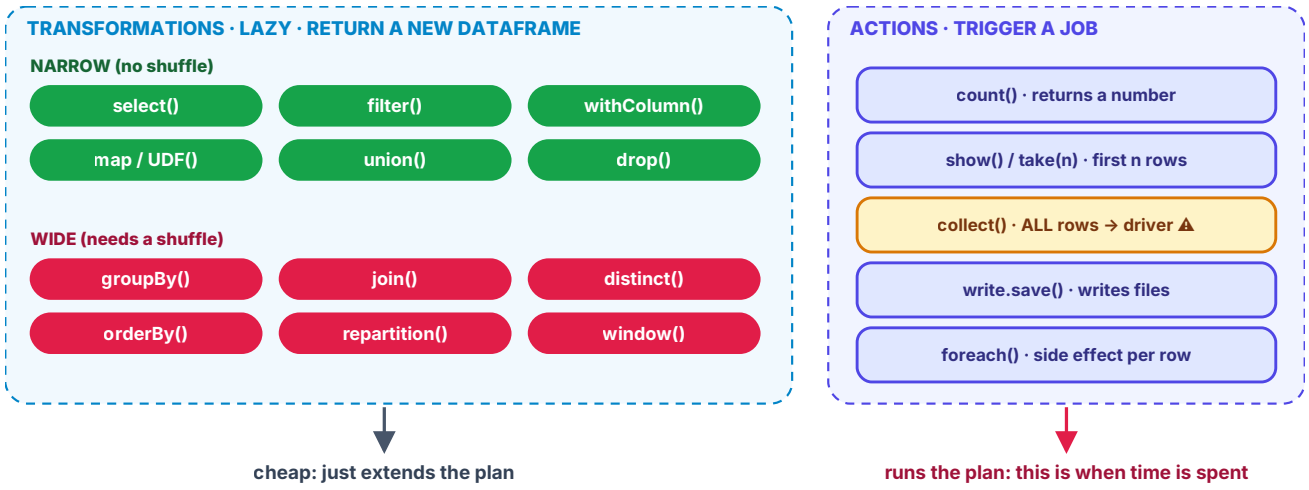


Figure 1.4 — Narrow and wide transformations, and the actions that run them

Call	Type	Notes
`select()` , `filter()` , `withColumn()`	Narrow transformation	Each output partition depends on one input partition: no data movement.
`groupBy().agg()` , `join()` , `distinct()`	Wide transformation	Rows with the same key must meet: **shuffle** (2.3).
`count()`	Action	Counts rows on executors, returns one number: safe.
`show()` / `take(n)`	Action	Reads only enough partitions to produce *n* rows.
`collect()` / `toPandas()`	Action	Pulls **everything** into driver memory. Dangerous on big data.
`write`	Action	Runs the plan on the cluster and writes in parallel.

COMMON TRAP

`df.collect()` or `df.toPandas()` on a large table is the textbook way to crash the driver (*OutOfMemoryError*). Aggregate first, then collect a small result, or write to storage instead.

INTERVIEW ANGLE

Likely question: *"Is `orderBy` a transformation or an action?"* A **wide transformation**: it is lazy, but triggers a shuffle (range partitioning) when the action runs.

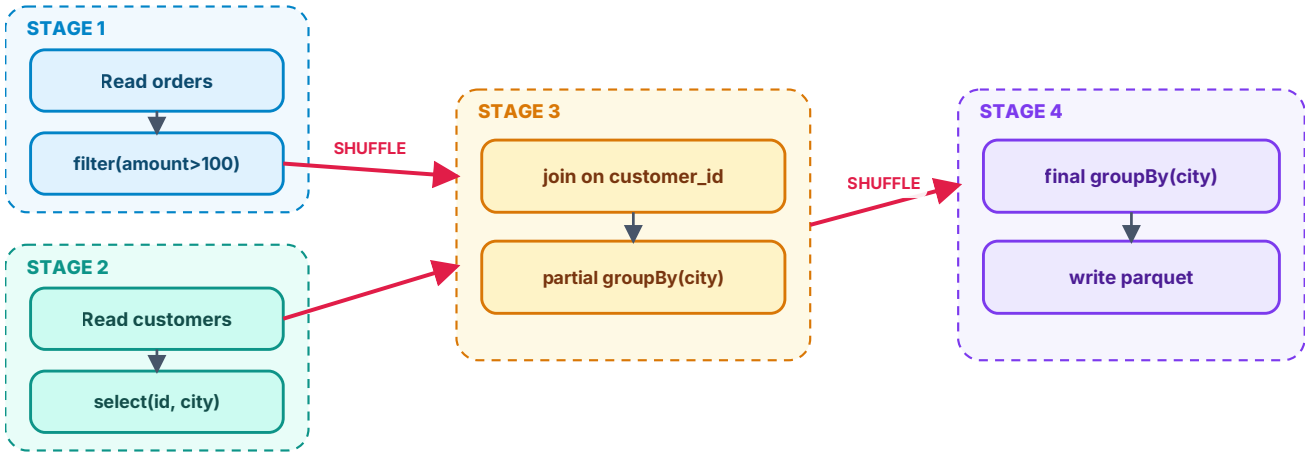
1.5 DAG (Directed Acyclic Graph)

★★★

IN ONE LINE

The graph of operations Spark builds, cut into stages at each shuffle.

The **DAG (Directed Acyclic Graph)** is Spark's map of the job: operations are nodes, data flow are arrows, and there are no cycles. The scheduler cuts the DAG into **stages** at every shuffle and runs each stage as parallel **tasks**.



Nodes = operations · arrows = data flow · red arrows = shuffles = stage boundaries

Directed: edges have a direction. Acyclic: no loops, so a plan can always be run start to finish.

Narrow steps (filter, select) are pipelined inside one stage; each shuffle cuts a new stage.

Figure 1.5 — A job's DAG split into four stages at shuffles

Lineage & fault tolerance

The DAG remembers how every partition was derived (its **lineage**). If an executor dies, Spark recomputes only the lost partitions from their parents — no replication needed.

Where you see it

Spark UI → **Jobs** → *DAG Visualization*, and **SQL / DataFrame** tab for the query plan graph (1.9).

REMEMBER

Fewer shuffles → fewer stages → faster job. Reading a DAG tells you **where data moves**, and data movement is what you optimise.

1.6 Jobs, Stages and Tasks

★★★

IN ONE LINE

Actions create jobs, shuffles split jobs into stages, partitions become tasks on cores.

Spark breaks every program into three levels of work. Knowing them lets you read the Spark UI and explain *why* something is slow.

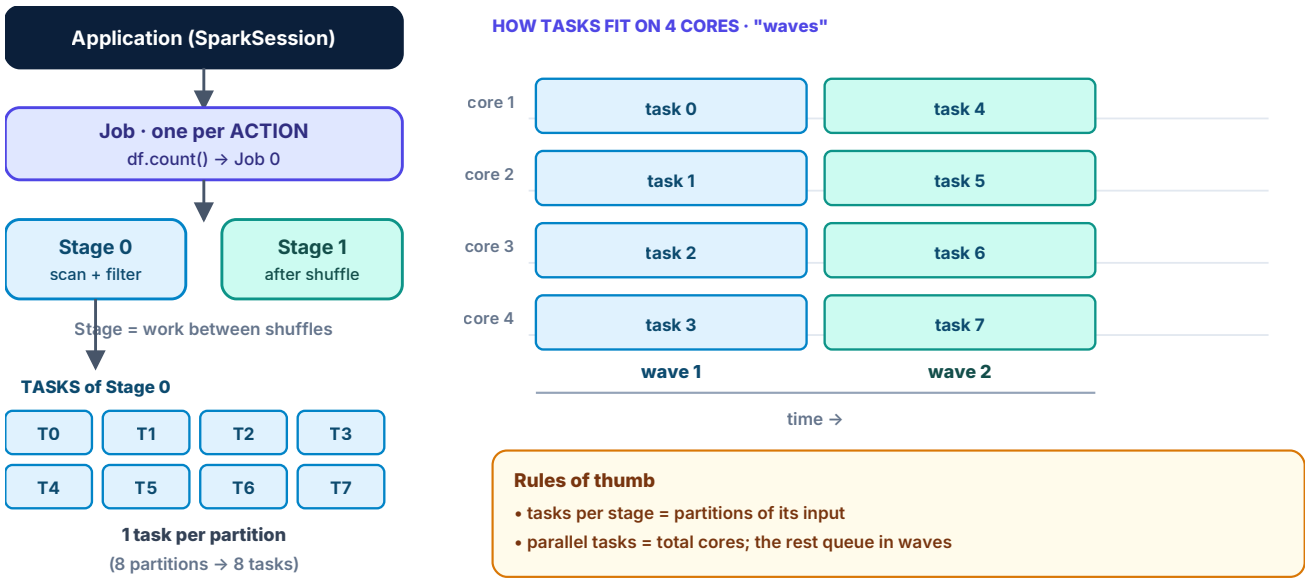


Figure 1.6 — Application → jobs → stages → tasks

Unit	Created by	Runs	Spark UI
Job	Each **action** (`count` , `write` , `show` ...)	A DAG of stages	Jobs tab
Stage	Split of a job at each **shuffle** boundary	A set of identical tasks	Stages tab
Task	One per **partition** of the stage	On one executor core	Stage detail → task table

EXAMPLE

A script with `df.count()` and then `df.groupBy("city").count().write…` has **2 jobs**. The second has **2 stages** (before and after the groupBy shuffle). If the input has 200 partitions, stage 1 has **200 tasks**; on 50 cores they run in 4 waves.

INTERVIEW ANGLE

Fast formula to quote: **tasks = partitions; concurrency = cores**. If you have 1000 cores and 10 partitions, 990 cores idle. If you have 10 cores and 100,000 tiny partitions, scheduling overhead dominates.

1.7 Logical and Physical Execution Plans

★★★

IN ONE LINE

A logical plan says what to compute; a physical plan says how, and explain() shows both.

Spark turns your query into a **logical plan** (what must happen), improves it, then chooses a **physical plan** (the concrete algorithms) and runs it. `explain()` lets you read these plans.

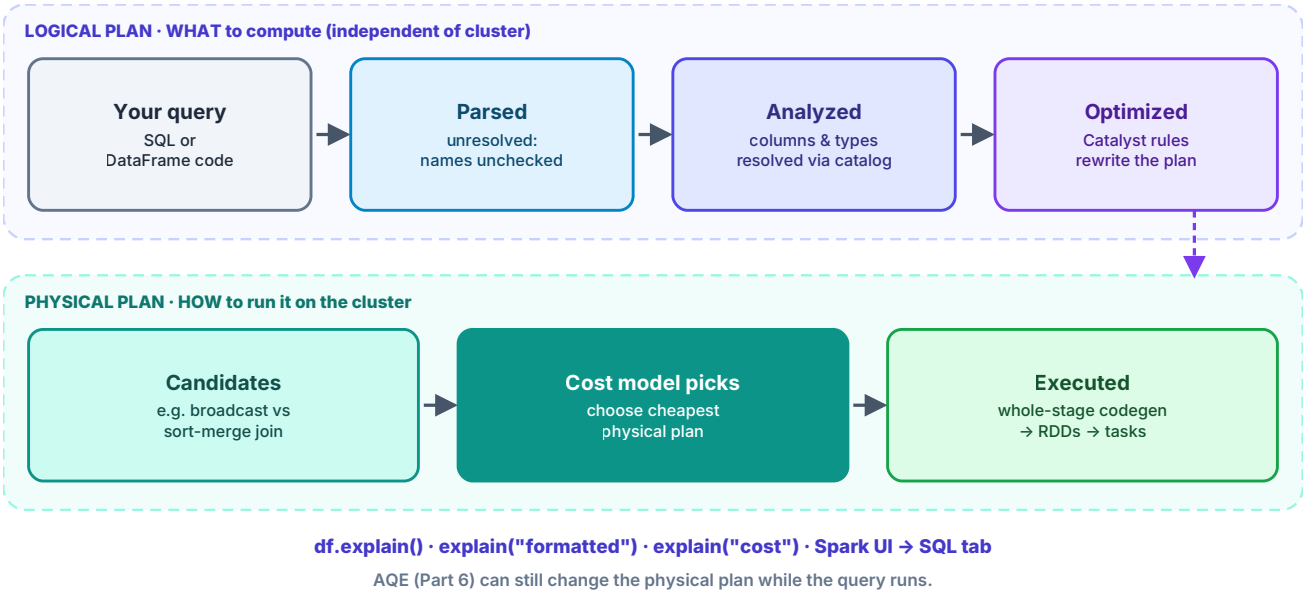


Figure 1.7 — From query to executed plan

Plan	Contains	Decided by
Unresolved logical	Parsed tree; names not yet checked	Parser
Analyzed logical	Real columns, types, tables	Analyzer + catalog
Optimized logical	Same result, cheaper shape	Catalyst rules (1.8)
Physical	Join algorithm, scan method, exchanges (shuffles)	Planner + cost model, then AQE

READ THE PLAN FROM THE BOTTOM UP

```
df = (orders.filter('amount > 100')
            .join(customers, 'customer_id')
            .groupBy('city').count())
df.explain('formatted')
```

SIMPLIFIED EXPLAIN OUTPUT: EXCHANGE = SHUFFLE; BROADCASTEXCHANGE = BROADCAST

```
== Physical Plan ==
AdaptiveSparkPlan
+- HashAggregate (final)           <- 4 final groupBy
   +- Exchange hashpartitioning(city)   <- SHUFFLE
      +- HashAggregate (partial)     <- 3 partial groupBy
         +- BroadcastHashJoin         <- 2 small table broadcast
            :- Filter (amount > 100)  <- 1 pushed to scan
            :  +- FileScan parquet [customer_id, amount]
            +- BroadcastExchange
```

INTERVIEW ANGLE

The two things to look for in `explain()` : **Exchange** nodes (shuffles) and the **join type**. This is how you *verify* an optimisation worked (6.5).

1.8 Catalyst Optimizer (and Tungsten)

★★★

IN ONE LINE

Catalyst automatically rewrites query plans; Tungsten makes execution memory- and CPU-efficient.

Catalyst is Spark SQL's rule-based and cost-based **query optimiser**. It rewrites your logical plan into an equivalent but cheaper one automatically — so DataFrame/SQL code is usually faster than hand-written RDD code. **Tungsten** is the execution engine underneath that makes the work itself CPU- and memory-efficient.

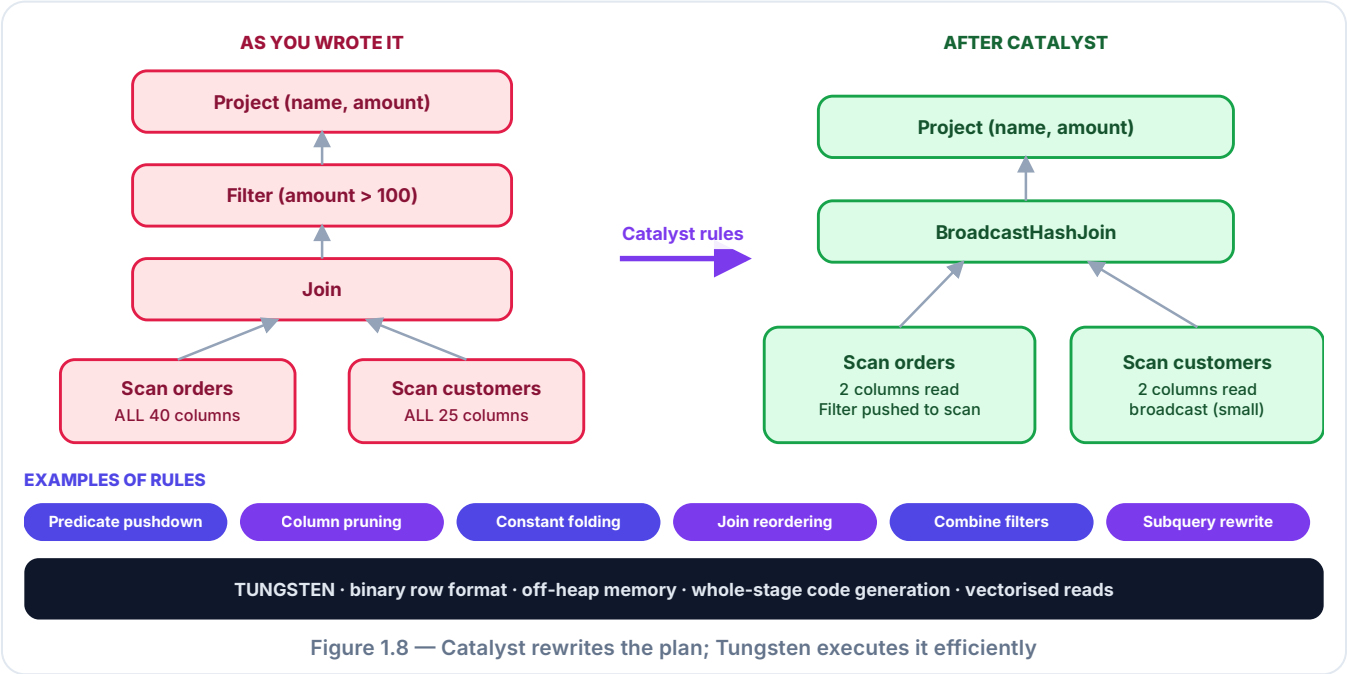


Figure 1.8 — Catalyst rewrites the plan; Tungsten executes it efficiently

Optimisation	What it does	Covered in
Predicate pushdown	Moves filters as close to the data source as possible	2.8
Column pruning	Reads only needed columns	2.7
Constant folding	Computes `1 + 2` once at planning time	—
Join selection / reordering	Chooses join algorithm and order using statistics	3.1, 3.3
Whole-stage codegen	Fuses several operators into one generated Java function	—

COMMON TRAP

Catalyst cannot see inside a **Python UDF** or an RDD lambda: they are black boxes, so filters cannot be pushed past them and codegen stops. Prefer built-in functions (3.8).

INTERVIEW ANGLE

Say it in one line: *"Catalyst optimises the **plan**; Tungsten optimises the **execution** (memory layout and generated code)."* Also: Catalyst's cost-based part needs table statistics (6.6).

1.9 Spark UI Basics

★★★

IN ONE LINE

The web UI that shows jobs, stages, tasks, executors and the query plan: your main debugging tool.

The **Spark UI** is your X-ray: it shows what actually happened. Every optimisation conversation ends with "what does the UI say?". Learn its tabs early.

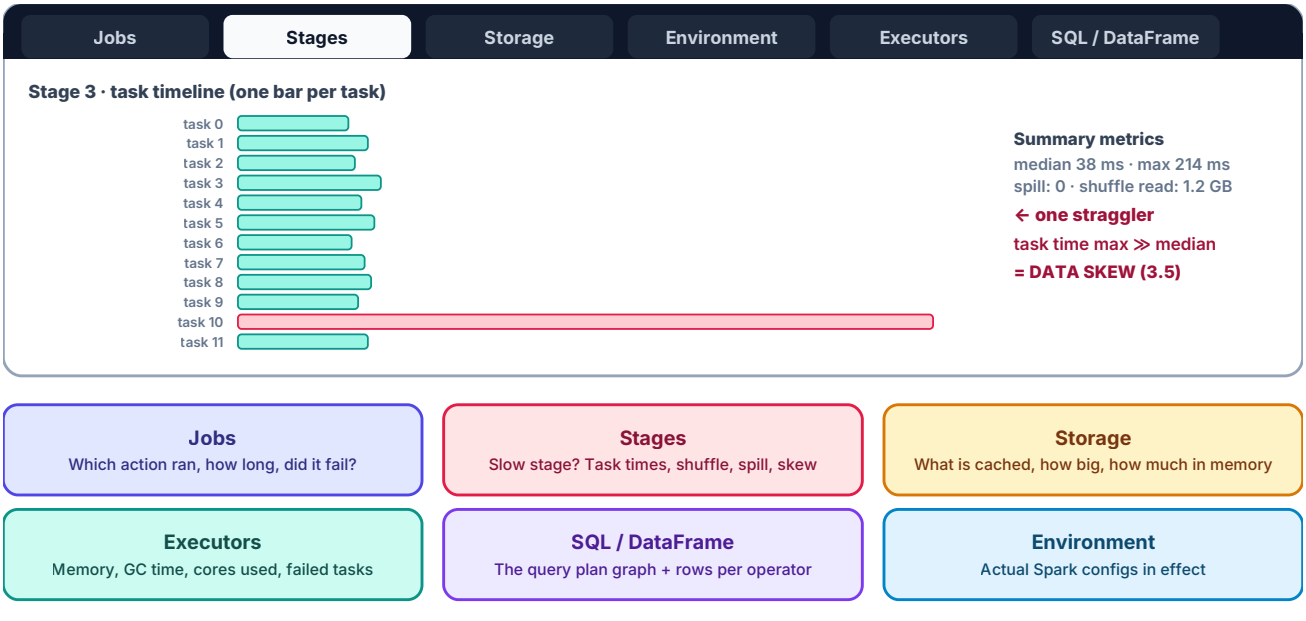


Figure 1.9 — Spark UI tabs and a stage timeline with a straggler

Look at	You learn	Red flag
Jobs / Stages	Duration of each stage, tasks succeeded / failed	One stage dominates total time
Stage detail	Task duration distribution, shuffle read/write, spill, GC time	Max task » median (skew); non-zero spill
SQL tab	Operator graph with row counts and times	Unexpected sort-merge join; huge row counts after a join
Executors	Memory use, GC time, active tasks, dead executors	High GC %, idle cores, lost executors
Storage	Cached DataFrames and their size / fraction cached	Cache partially on disk or not fitting

INTERVIEW ANGLE

Interviewers love: *"How do you find the bottleneck?"* Answer: **Jobs → slowest stage → task metrics (skew, spill, shuffle) → SQL tab to see the operator**. Full method in Part 9.